

Université Ibn Tofail
École Nationale des Sciences Appliquées de Kénitra

RAPPORT DE PROJET DE FIN D'ANNÉE

Cycle ingénieur

GlassInk Monitor

Conception d'une plateforme IoT de supervision d'une imprimante industrielle CIJ

Application à une ligne de marquage de pare-brise automobile

Réalisé par : Omar Bengourich

Élève ingénieur en Réseaux et Systèmes de Communication
Option Systèmes ubiquitaires

Organisme d'accueil : Saint-Gobain Sekurit Maroc

Dédicace

À mes parents et à ma famille, pour leur confiance et leur patience pendant mes années d'études. À mes amis et à mes camarades de promotion, avec qui les problèmes techniques ont souvent fini par devenir des souvenirs. Je leur dédie ce travail.

Remerciements

Je remercie les équipes de Saint-Gobain Sekurit Maroc qui ont partagé leur connaissance de la ligne de production et du contexte industriel. Leurs explications m'ont permis de comprendre qu'un arrêt d'imprimante ne se résume pas à un voyant rouge : il touche la cadence, la qualité du marquage et le travail de l'équipe de maintenance.

Je remercie également les enseignants de l'École Nationale des Sciences Appliquées de Kénitra pour la formation reçue en réseaux, systèmes de communication et informatique industrielle. Enfin, je remercie ma famille et mes proches pour leur soutien pendant la préparation de ce projet.

Résumé

Ce projet porte sur la supervision d'une imprimante à jet d'encre continu (CIJ) utilisée pour le marquage de vitrages automobiles. Les données de la machine et de la ligne sont déjà collectées au niveau de Litmus Edge. Le travail consiste à construire la chaîne située en aval : transport MQTT, ingestion visuelle avec Node-RED, stockage temporel dans InfluxDB, tableaux de bord Grafana et alertes.

Le prototype suit l'état de l'imprimante, les niveaux d'encre et de solvant, la viscosité, les températures, les compteurs, les défauts et les dates de péremption des consommables. Une difficulté particulière vient du fonctionnement à la commande. La production est possible 24 heures sur 24, cinq jours sur sept, mais la machine peut rester inactive lorsqu'aucun pare-brise de la référence concernée n'est demandé. Le calcul de disponibilité utilise donc un signal de demande et ne confond pas l'absence de commande avec une panne.

Faute d'accès au Litmus Edge réel pendant le développement, un simulateur reproduit les états, les consommations, les périodes sans commande et les défauts d'une imprimante CIJ. Les essais ont validé le parcours complet des données, le rejet des messages invalides, la reprise après redémarrage et la protection contre les doublons MQTT.

Mots-clés : Internet des objets industriel, CIJ, MQTT, Node-RED, InfluxDB, Grafana, maintenance, pare-brise.

Abstract

This project addresses the monitoring of a continuous inkjet (CIJ) printer used to mark automotive glazing. Machine and production-line data are already collected by Litmus Edge. The work focuses on the downstream platform : MQTT transport, visual ingestion with Node-RED, InfluxDB time-series storage, Grafana dashboards, and alert delivery.

The prototype monitors printer state, ink and solvent levels, viscosity, temperatures, counters, faults, and consumable expiry dates. Availability cannot be measured against wall-clock time because production is order-driven. A demand signal separates actual downtime from normal idle periods without an order.

Since the real Litmus Edge was not available during development, a simulator reproduces CIJ behavior, fluid consumption, order gaps, and injected faults. End-to-end tests validated data persistence, malformed-message handling, restart recovery, and MQTT duplicate protection.

Keywords : Industrial Internet of Things, CIJ, MQTT, Node-RED, InfluxDB, Grafana, maintenance, automotive glazing.

Liste des sigles

CIJ	Continuous Inkjet, impression à jet d'encre continu.
DMC	Data Matrix Code, code bidimensionnel de traçabilité.
IIoT	Industrial Internet of Things.
MQTT	Message Queuing Telemetry Transport.
OT	Operational Technology, systèmes industriels de production.
IT	Information Technology, systèmes informatiques de gestion.
PLC / API	Programmable Logic Controller / automate programmable industriel.
PROFINET	Protocole Ethernet industriel utilisé sur la ligne.
QoS	Quality of Service, niveau de garantie de livraison MQTT.
TRS	Taux de rendement synthétique.

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des sigles	v
1 Introduction générale	1
1.1 Point de départ	1
1.2 Objectifs	1
1.3 Démarche suivie	2
1.4 Organisation du rapport	2
2 Saint-Gobain Sekurit et le contexte industriel	3
2.1 Présentation du site de Kénitra	3
2.2 Le pare-brise feuilleté	4
2.3 Place de l'impression et du contrôle	4
2.4 Systèmes industriels présents	5
3 Recherche bibliographique et état de l'art	6
3.1 Démarche de recherche	6
3.2 De la maintenance corrective à la surveillance conditionnelle	6
3.3 Données disponibles sur la Markem-Imaje 9450c	7
3.4 Acquisition et interopérabilité à la frontière OT/IT	8
3.5 Transport des données	8
3.6 Solutions d'ingestion comparées	8
3.7 Stockage, visualisation et alertes	9
3.8 Synthèse et positionnement du projet	10
3.9 Problématique retenue	11
4 Analyse du besoin et cahier des charges	12

4.1	Situation observée	12
4.2	Parties prenantes	12
4.3	Exigences fonctionnelles	12
4.4	Exigences non fonctionnelles	13
4.5	Périmètre et hypothèses	14
4.6	Choix d'un simulateur	14
5	Conception de la solution	15
5.1	Architecture générale	15
5.2	Choix de la pile technique	15
5.2.1	MQTT et Mosquitto	16
5.2.2	Node-RED	16
5.2.3	InfluxDB	16
5.2.4	Grafana	16
5.2.5	Docker Compose	16
5.3	Cycle de vie des données	17
5.4	Topics MQTT	17
5.5	Modèle de données	18
5.6	Disponibilité liée à la demande	19
5.7	Prévision et péremption	19
6	Réalisation de la plateforme	21
6.1	Organisation du projet	21
6.2	Simulateur de l'imprimante CIJ	21
6.3	Ingestion avec Node-RED	22
6.4	Écriture idempotente et reprise	24
6.5	Requêtes Flux	24
6.6	Conteneurisation	25
6.7	Notification des alertes	25
7	Tableaux de bord et alertes	26
7.1	Principes de présentation	26
7.2	Vue opérateur	26
7.3	Vue maintenance	27
7.4	Vue direction	28
7.5	Règles d'alerte	30
7.6	Lisibilité et langue	31
8	Validation du prototype	32
8.1	Stratégie de test	32
8.2	Tests unitaires	32
8.3	Essais d'intégration	33

8.4	Cas de la disponibilité	33
8.5	Cas des consommables	34
8.6	Limites de la validation	34
9	Sécurité et préparation du déploiement	35
9.1	Constat réseau	35
9.2	Mesures intégrées au prototype	35
9.3	Mesures requises sur site	36
9.4	Politique MQTT	36
9.5	Données et conservation	36
10	Conclusion générale	38
A	Contrats de messages	39
A.1	Exemple de télémétrie	39
A.2	Exemple d'événement de remplissage	39
B	Commandes d'exploitation	41
	Bibliographie	42

Table des figures

2.1	Vue du site Saint-Gobain Sekurit de Kénitra	3
2.2	Produit et chaîne de fabrication du vitrage automobile	4
2.3	Exemples d’impression et de contrôle dans la ligne	4
3.1	Imprimante CIJ Markem-Imaje 9450 et tête de marquage	7
5.1	Architecture fonctionnelle de GlassInk Monitor	15
5.2	Traitement d’une mesure entre la machine et le tableau de bord	17
5.3	Organisation des données dans le bucket InfluxDB	18
5.4	Distinction entre production, arrêt et attente normale	19
6.1	Chaîne d’ingestion mise en œuvre avec Node-RED	22
6.2	Flux Node-RED de télémétrie : MQTT, validation et écriture InfluxDB . .	23
6.3	Flux Node-RED de supervision et de traitement des rejets	24
7.1	Tableau de bord opérateur alimenté par la simulation	27
7.2	Paramètres natifs produits en direct par la simulation 9450c	28
7.3	Tableau de bord de maintenance alimenté par la simulation 9450c	29
7.4	Tableau de bord de direction alimenté par les données simulées	30
9.1	Déploiement proposé et mesures à appliquer avant le raccordement . . .	35

Liste des tableaux

2.1	Fiche synthétique de l'organisme d'accueil	3
3.1	Comparaison des solutions d'ingestion	9
3.2	Positionnement de GlassInk Monitor	10
4.1	Attentes des utilisateurs	12
4.2	Exigences fonctionnelles du prototype	13
5.1	Arborescence MQTT du prototype	17
6.1	Comportements reproduits par le simulateur	22
7.1	Correspondance entre état technique et texte affiché	26
7.2	Règles provisionnées dans Grafana	30
8.1	Résultats des essais de bout en bout	33
B.1	Commandes principales du prototype	41

Introduction générale

1.1 Point de départ

Une imprimante industrielle peut paraître secondaire à côté d'un four, d'un robot de chargement ou d'une ligne d'assemblage. Pourtant, si elle ne marque plus correctement le verre, la pièce perd une partie de sa traçabilité et la production peut être arrêtée. Dans le cas d'une imprimante à jet d'encre continu, les causes sont variées : niveau de solvant trop bas, viscosité hors plage, défaut de pression, buse obstruée ou consommable périmé.

Sur la ligne étudiée, les automates communiquent en PROFINET et les données remontent déjà vers Litmus Edge. Elles restent toutefois difficiles à exploiter par les personnes qui en ont besoin. L'opérateur veut connaître l'état immédiat de la machine. La maintenance veut comprendre les dérives et les défauts récurrents. La direction regarde surtout la disponibilité et le temps perdu. Ces trois besoins ne se traduisent pas par le même écran.

Le chapitre consacré à l'état de l'art situe cette difficulté parmi les approches de supervision industrielle avant de formuler la problématique retenue.

1.2 Objectifs

Le projet poursuit les objectifs suivants :

- afficher l'état courant de l'imprimante et le contexte de production ;
- suivre l'encre, le solvant, la viscosité, les températures et les heures de fonctionnement ;
- estimer le temps restant avant épuisement d'un consommable ;
- tenir compte de la date limite d'utilisation et de la durée après ouverture ;
- enregistrer les défauts et calculer le temps d'arrêt seulement lorsque la ligne demande une impression ;
- offrir des tableaux de bord distincts pour l'opérateur, la maintenance et la direction ;
- valider l'ensemble sur un simulateur réaliste avant le raccordement au site.

1.3 Démarche suivie

La démarche repose sur une séparation claire entre le réseau industriel et la plateforme de supervision. Litmus Edge constitue le point d'entrée des données. En aval, une architecture conteneurisée assure le transport MQTT, la validation des messages, leur stockage temporel et leur présentation dans Grafana.

Le développement a été mené avec un simulateur d'imprimante CIJ afin de disposer de scénarios reproductibles : production normale, attente sans commande, consommation des fluides, dérive de viscosité, remplissage et défaut. La même chaîne pourra ensuite recevoir les messages de Litmus Edge en remplaçant le mapping d'entrée, sans modifier les tableaux de bord.

1.4 Organisation du rapport

Le chapitre suivant présente Saint-Gobain Sekurit Maroc et le procédé de fabrication dans lequel s'inscrit le marquage. Le chapitre 3 expose la recherche bibliographique, l'état de l'art et la problématique. Le chapitre 4 décrit le besoin fonctionnel et les contraintes. Les chapitres 5 et 6 détaillent la conception et la réalisation. Le chapitre 7 traite des tableaux de bord et des alertes. Les essais sont regroupés au chapitre 8. Le chapitre 9 revient sur la sécurité et les conditions du raccordement industriel. Le rapport se termine par un bilan et les travaux restant à effectuer sur site.

Saint-Gobain Sekurit et le contexte industriel

2.1 Présentation du site de Kénitra

Saint-Gobain Sekurit fabrique des vitrages automobiles pour la première monte et le marché du remplacement. Implanté dans l'Atlantic Free Zone, le site de Kénitra produit des pare-brise automobiles et dessert plusieurs constructeurs. Son organisation associe les opérations de transformation du verre, le marquage, l'assemblage et le contrôle qualité.



FIGURE 2.1 – Vue du site Saint-Gobain Sekurit de Kénitra

TABLE 2.1 – Fiche synthétique de l'organisme d'accueil

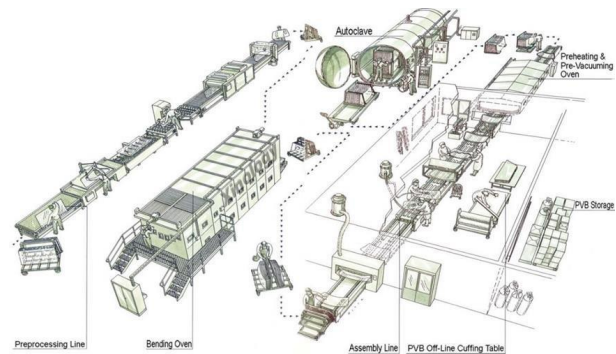
Élément	Présentation synthétique
Entreprise	Saint-Gobain Sekurit Maroc
Implantation	Atlantic Free Zone, Kénitra
Activité	Production de pare-brise automobiles
Clients cités	Stellantis, Renault, Volkswagen et Seat
Certifications citées	IATF 16949, ISO 9001, ISO 14001 et ISO 45001
Produits présentés	Pare-brise feuilleté, PVB chauffant et PVB antenne

2.2 Le pare-brise feuilleté

Un pare-brise feuilleté associe deux feuilles de verre et un film intermédiaire en PVB. Ce film maintient les fragments en cas de choc et peut recevoir des fonctions supplémentaires, comme le chauffage ou une antenne. La fabrication ne consiste donc pas seulement à découper une forme. Elle enchaîne des opérations de préparation, d'impression, de bombage, d'assemblage et de contrôle [1].



(a) Pare-brise feuilleté

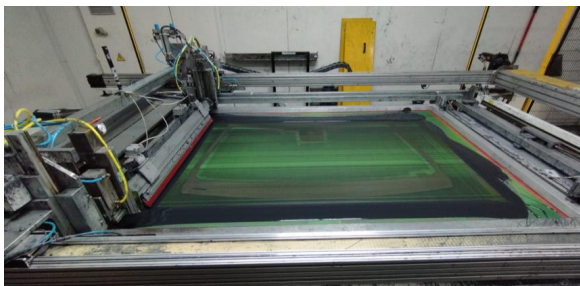


(b) Vue d'ensemble du procédé

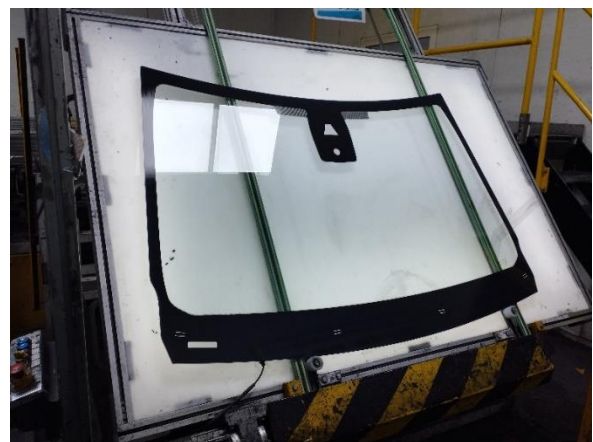
FIGURE 2.2 – Produit et chaîne de fabrication du vitrage automobile

2.3 Place de l'impression et du contrôle

Dans la partie pré-processing, le verre est chargé, découpé, façonné, lavé et imprimé. La sérigraphie du logo et de la bande noire est suivie d'un contrôle visuel et d'un contrôle automatique. Le projet GlassInk Monitor concerne un autre équipement d'impression, une imprimante CIJ dédiée au marquage DMC, mais les enjeux restent proches : position correcte, contraste suffisant, lisibilité et traçabilité.



(a) Table d'impression sur verre



(b) Pare-brise au poste de contrôle

FIGURE 2.3 – Exemples d'impression et de contrôle dans la ligne

Un défaut de marquage peut avoir plusieurs origines. La donnée imprimée peut

être fausse, la machine peut manquer de fluide, la viscosité peut dériver ou la buse peut être partiellement obstruée. Une supervision utile doit donc réunir les informations du procédé, de l'imprimante et des consommables.

2.4 Systèmes industriels présents

Les PLC coordonnent les équipements de la ligne et échangent leurs données sur PROFINET. Litmus Edge récupère les tags industriels, les normalise et peut les publier vers des systèmes informatiques. Il ne remplace pas les automates : les automatismes restent responsables du contrôle de la production.

Ce découpage fixe la frontière du projet. GlassInk Monitor commence au niveau de la sortie MQTT de Litmus Edge. La configuration des drivers PLC et la logique de commande de la ligne ne font pas partie du travail.

Recherche bibliographique et état de l'art

3.1 Démarche de recherche

Cette étude ne prétend pas être une revue systématique de toute la littérature sur l'IIoT. Elle répond à une question plus concrète : quelles solutions permettent de construire une supervision industrielle à partir de données déjà disponibles sur une plateforme edge ? La recherche a donc suivi six axes : l'interopérabilité OT/IT, la surveillance conditionnelle, le transport MQTT, l'ingestion des messages, le stockage temporel et la visualisation.

Les sources ont été choisies dans un ordre simple. Les normes et spécifications officielles ont servi pour les protocoles. Les documentations des éditeurs ont permis de vérifier les possibilités des outils. Enfin, des articles scientifiques ont été consultés pour situer le projet par rapport à la maintenance conditionnelle et prédictive. Les critères de sélection ont été la compatibilité avec Litmus Edge, la possibilité d'un déploiement local, la lisibilité pour une équipe industrielle et un coût raisonnable pour une seule ligne.

3.2 De la maintenance corrective à la surveillance conditionnelle

La maintenance corrective intervient après la panne. La maintenance préventive remplace ou contrôle un équipement selon un calendrier, même si son état réel ne l'exige pas encore. La maintenance conditionnelle s'appuie au contraire sur des mesures : température, pression, vibration, niveau, viscosité ou compteur d'heures. Elle cherche une dérive avant que le défaut ne bloque la production.

La maintenance prédictive va plus loin. Elle utilise un historique suffisant pour estimer une probabilité de panne ou une durée de vie restante. La revue de Krupitzer et al. montre que cette démarche repose sur la collecte, la préparation et l'interprétation de données industrielles, souvent avec des méthodes d'apprentissage automatique [18]. Li, Fei et Zhang décrivent une architecture edge-to-cloud appliquée à la surveillance d'équipements de fabrication. Le traitement à la périphérie réduit le volume échangé et rapproche l'analyse de la machine [17].

GlassInk Monitor se place volontairement au niveau de la **surveillance conditionnelle**. Le système détecte des seuils, suit des tendances et extrapole le temps avant épuisement d'un fluide. Il ne prédit pas encore une panne par apprentissage automatique.

Il manque pour cela des mois de données réelles, des défauts correctement étiquetés et un retour de la maintenance. Employer dès maintenant le terme de maintenance prédictive donnerait une image plus ambitieuse, mais techniquement inexacte.

3.3 Données disponibles sur la Markem-Imaje 9450c

La ligne étudiée utilise une imprimante Markem-Imaje 9450c à jet d'encre continu, adaptée aux cadences industrielles et équipée d'une connectivité Ethernet [2]. Elle est raccordée à un automate Siemens S7-1515-2 PN. La communication s'appuie sur TCP/IP et sur des trames contenant un identifiant, une longueur, des données utiles et une somme de contrôle.



FIGURE 3.1 – Imprimante CIJ Markem-Imaje 9450 et tête de marquage

Le périmètre de supervision retient huit familles d'informations : alertes, état général, état de l'impression, état du jet, informations des deux cartouches, paramètres de fonctionnement et tâche d'impression active.

La réponse des paramètres de fonctionnement contient 84 octets et plus de quarante valeurs. Parmi elles figurent la vitesse moteur, la pression, la viscosité, le niveau d'encre, la quantité d'additif injectée, les températures électronique, encre et tête, les états des électrovannes, le niveau du tube, l'autonomie d'encre et la consommation moyenne. Les cartouches externes d'encre et d'additif ont une capacité de 800 mL ; les réservoirs internes ont une capacité annoncée de 1,075 L et maintiennent une réserve de fonctionnement d'environ 24 heures. La capacité des cartouches externes et les fonctions de suivi de l'autonomie sont également indiquées dans la documentation technique du constructeur [2].

Dans l'architecture existante, la 9450c est reliée à Logoface4, puis au système Gosun. Les données rejoignent ensuite Litmus Edge et l'IoT Backbone. Cette organisation confirme que la plateforme de supervision n'a pas à interroger directement l'imprimante :

les données déjà centralisées sur Litmus Edge constituent son point d'entrée. Elle fournit aussi une base concrète au simulateur, qui reproduit les codes d'état du jet, l'état d'impression et les paramètres utiles à la maintenance.

3.4 Acquisition et interopérabilité à la frontière OT/IT

Dans une architecture industrielle classique, OPC UA offre un modèle d'information commun, des services client–serveur et un mode publication–abonnement. La spécification prévoit aussi l'authentification, le chiffrement et le contrôle d'intégrité [7]. Une application pourrait donc lire directement les variables d'un automate ou d'un serveur OPC UA.

Ce choix n'est pas retenu ici. Les automates, l'imprimante et les équipements de la ligne sont déjà raccordés à Litmus Edge. La plateforme communique avec les équipements, conserve des données localement et peut les publier vers MQTT ou REST [6]. Ajouter un second client direct vers les PLC dupliquerait la collecte et aggraverait inutilement le périmètre de sécurité. Le point d'entrée de GlassInk Monitor est donc la sortie nord de Litmus Edge. L'application reste en lecture seule et ne commande aucun équipement.

3.5 Transport des données

MQTT découple la source et les consommateurs. Litmus Edge publie sans connaître Node-RED, et Node-RED peut redémarrer sans modifier la configuration des automates. Le QoS 1 retenu garantit une livraison au moins une fois, ce qui signifie qu'un message peut être reçu deux fois [3]. Cette propriété explique la protection contre les doublons mise en place dans la base.

MQTT ne définit cependant ni le nom des topics industriels ni la structure du contenu. Sparkplug complète MQTT avec un espace de noms, un format de charge utile et une gestion de l'état de session adaptée aux usages IIoT [8]. Son format binaire Protobuf demande néanmoins un décodage supplémentaire.

Pour le prototype, des topics explicites et un contenu JSON ont été choisis. Ils sont faciles à lire, à simuler et à contrôler pendant une démonstration. Lors du raccordement, deux cas restent possibles : Litmus Edge publie ce JSON, ou il publie en Sparkplug B et un décodeur est ajouté à l'entrée du flux. Cette adaptation ne change ni le modèle des mesures ni les tableaux de bord.

3.6 Solutions d'ingestion comparées

L'ingestion ne consiste pas seulement à copier un message. Elle vérifie le schéma, sépare télémétrie et événements, rejette les données invalides et garantit l'idempotence. Trois approches ont été étudiées.

TABLE 3.1 – Comparaison des solutions d’ingestion

Solution	Points forts	Limites dans ce projet	Appréciation
Telegraf	Configuration déclarative, nombreux plugins, faible consommation.	Moins souple pour les événements hétérogènes, les remplissages et les transactions métier.	Très adapté à des mesures régulières et déjà normalisées.
Node-RED	Flux visuel, adaptation rapide des messages, bonne lisibilité du chemin de données.	L’éditeur doit être protégé et les flux doivent rester versionnés et testés.	Retenu pour le prototype et le raccordement à Litmus Edge.
Service spécifique	Contrôle complet du décodage et des transactions.	Davantage de code, de maintenance et une logique moins visible pour l’équipe industrielle.	À réserver à un besoin que Node-RED ne peut pas couvrir proprement.

Telegraf reste une bonne solution lorsque les messages sont déjà homogènes et que le chemin entre la source et la base demande peu d’adaptation [10]. Ici, la télémétrie, les événements de maintenance et les résultats de marquage n’ont pas la même structure. Node-RED a donc été retenu pour valider chaque famille, ajouter les tags utiles et produire le Line Protocol attendu par InfluxDB. Le flux reste lisible dans l’éditeur tout en étant conservé dans le dépôt du projet.

3.7 Stockage, visualisation et alertes

InfluxDB organise une donnée autour d’une mesure, de tags indexés, de champs et d’un horodatage [11, 12]. Ce modèle correspond directement aux variables de l’imprimante : l’identité de la machine et la ligne deviennent des tags, tandis que les niveaux, températures et compteurs restent des champs. Les défauts et les marquages sont conservés dans des mesures distinctes. Cette séparation évite de multiplier les colonnes lorsque la liste des tags Litmus évolue.

Le prototype utilise InfluxDB OSS 2.7 avec une organisation et un bucket dédiés. La durée de rétention du bucket est illimitée pour la démonstration ; elle devra être fixée avec les équipes qualité et infrastructure avant le raccordement au site. Node-RED enrichit chaque point avec la classification de la machine et les prévisions de consommables. Les requêtes Flux filtrent ensuite les séries, appliquent les fenêtres temporelles et agrègent les indicateurs affichés dans Grafana [14].

Grafana dispose d’une source InfluxDB native. Elle interroge le bucket en Flux, construit les vues opérateur, maintenance et direction, puis évalue les règles d’alerte [15]. Pour une ligne et moins de 200 pièces par heure, une instance de chaque service suffit. La haute disponibilité pourra être étudiée à partir de mesures réelles de charge et des exigences du site.

3.8 Synthèse et positionnement du projet

TABLE 3.2 – Positionnement de GlassInk Monitor

Brique	Options étudiées	Choix	Motif principal
Point d'intégration	PLC/OPC UA direct, Litmus Edge	Litmus Edge	La collecte industrielle existe déjà.
Transport	MQTT JSON, Sparkplug B	MQTT JSON au prototype	Lecture et simulation simples ; décodage Sparkplug ajoutable.
Ingestion	Telegraf, Node-RED, service spécifique	Node-RED	Transformation visuelle des mesures et événements.
Stockage	InfluxDB, base relationnelle, stockage par fichiers	InfluxDB 2.7	Modèle temporel natif, rétention et intégration directe avec Grafana.
Supervision	Application web, Grafana	Grafana	Tableaux de bord, rôles et alertes déjà disponibles.
Déploiement	Services natifs, Docker Compose, Kubernetes	Docker Compose	Reproductible et proportionné à une seule ligne.
Niveau d'analyse	Seuils, surveillance conditionnelle, modèle prédictif	Surveillance conditionnelle	Pas encore d'historique réel étiqueté pour entraîner un modèle.

L'état de l'art conduit ainsi à une architecture edge-to-dashboard maîtrisée : Litmus Edge comme frontière industrielle, MQTT pour le découplage, Node-RED pour l'adaptation, InfluxDB pour l'historisation et Grafana pour l'usage quotidien. L'apport du projet tient à la manière dont ces briques répondent au fonctionnement de la ligne : l'activité dépend de la commande, les fluides peuvent expirer avant d'être vides et le raccordement final doit rester limité à la configuration de la source Litmus Edge.

3.9 Problématique retenue

À la suite de cette analyse, la problématique du PFA est formulée ainsi :

Comment transformer les données disponibles sur Litmus Edge en une supervision fiable de l'imprimante CIJ, capable d'anticiper les ruptures et la péremption des consommables, de distinguer un arrêt réel d'une période sans commande et de présenter une information adaptée à chaque utilisateur ?

Cette question générale se décline en trois questions d'ingénierie :

1. comment transporter et enregistrer les messages en tenant compte des retransmissions possibles avec le QoS 1 ?
2. comment calculer la disponibilité sans classer une absence de commande comme une panne ?
3. comment réunir dans une seule décision le temps avant épuisement et la date effective de péremption d'un fluide ?

Deux contraintes encadrent la réponse. Le projet ne commande ni l'imprimante ni les automates : le flux reste en lecture seule. Par ailleurs, l'accès au Litmus Edge réel n'était pas disponible pendant le développement. La solution devait donc être construite et vérifiée avec un simulateur, puis rester remplaçable par la source industrielle sans reprendre toute l'application.

Analyse du besoin et cahier des charges

4.1 Situation observée

La ligne peut produire 24 heures sur 24, cinq jours par semaine. Cela ne veut pas dire que l'imprimante doit fonctionner en continu. Son utilisation dépend des commandes et de la référence de pare-brise en cours. Pendant une période sans demande, une imprimante arrêtée n'est pas en panne. À l'inverse, si un verre arrive au poste et que l'imprimante est indisponible, le temps perdu doit être enregistré.

Cette différence paraît simple, mais elle change les indicateurs. Une disponibilité calculée sur le temps calendaire pénaliserait les périodes sans commande. Elle produirait des alertes inutiles, puis les utilisateurs finiraient par les ignorer.

4.2 Parties prenantes

TABLE 4.1 – Attentes des utilisateurs

Utilisateur	Question principale	Information attendue
Opérateur	La machine peut-elle imprimer maintenant ?	État, niveau des fluides, défaut actif et volume récent.
Maintenance	Pourquoi la machine s'arrête-t-elle et quand intervenir ?	Tendances, heures restantes, historique, Pareto des défauts et service à prévoir.
Direction	Quel est l'effet sur la production ?	Disponibilité liée à la demande, temps d'arrêt, rendement et volume marqué.

4.3 Exigences fonctionnelles

TABLE 4.2 – Exigences fonctionnelles du prototype

Réf.	Exigence	Critère d'acceptation
F01	Recevoir les mesures et événements par MQTT.	Les trois familles de topics sont consommées.
F02	Afficher l'état courant de l'imprimante.	État mis à jour en moins d'une période de publication.
F03	Suivre les niveaux d'encre et de solvant.	Courbes, valeur courante et seuils visibles.
F04	Estimer le temps avant épuisement.	Prévision calculée à partir d'une pente récente.
F05	Gérer la péremption d'un consommable.	Date imprimée et durée après ouverture comparées.
F06	Distinguer un arrêt réel d'une absence de commande.	Le signal de demande conditionne l'indicateur.
F07	Enregistrer les défauts et leur durée.	Historique filtrable et Pareto sur 30 jours.
F08	Conserver le marquage DMC et son résultat de contrôle.	Recherche par DMC et statistiques de qualité possibles.
F09	Déclencher des alertes.	Sept règles provisionnées et un webhook testé.
F10	Présenter des vues adaptées aux trois publics.	Trois tableaux de bord distincts.

4.4 Exigences non fonctionnelles

- **Lecture seule** : aucun service du projet n'écrit vers le PLC ou l'imprimante.
- **Reproductibilité** : l'environnement complet doit démarrer avec Docker Compose.
- **Traçabilité** : chaque message possède un identifiant unique.

- **Résistance aux doublons** : une livraison MQTT répétée ne doit pas créer deux mesures ou deux événements.
- **Observabilité** : chaque service expose un contrôle de santé ou un état visible dans Docker.
- **Confidentialité** : les mots de passe ne figurent ni dans les flux Node-RED ni dans le fichier Compose.
- **Évolutivité raisonnable** : l'ajout d'un tag ne doit pas imposer une refonte du modèle de stockage.

4.5 Périmètre et hypothèses

Le périmètre s'arrête à la supervision. Les commandes de production, l'ERP, le MES, le réglage de l'imprimante et l'écriture dans les automates sont exclus. Le prototype utilise les hypothèses suivantes :

- une seule ligne, avec moins de 200 pare-brise par heure ;
- une imprimante CIJ Markem-Imaje 9450c ;
- un signal `line_demanding` indiquant qu'une impression est attendue ;
- une caméra de contrôle fournissant un résultat et un grade ;
- une durée d'utilisation de 90 jours après ouverture pour les fluides.

Ces valeurs servent au développement. Le modèle exact de l'imprimante, la liste des tags et la durée réelle après ouverture devront être confirmés avec la documentation du site avant la mise en service.

4.6 Choix d'un simulateur

Attendre l'accès au Litmus Edge aurait bloqué toutes les étapes suivantes. Le simulateur n'est donc pas un simple générateur de nombres aléatoires. Il reproduit les situations qui peuvent tromper la supervision : démarrage, préchauffage, production, attente sans commande, jet laissé actif, défaut, remplissage, dérive de viscosité et perte temporaire de communication.

Il accélère le temps par un facteur configurable. Une minute réelle peut représenter une heure simulée. Les tendances et les alertes deviennent ainsi visibles pendant une démonstration, tout en gardant des horodatages réels dans la base.

Conception de la solution

5.1 Architecture générale

L'architecture sépare la collecte industrielle, le transport, le traitement, le stockage et la visualisation. Cette séparation facilite les essais et permet de remplacer le simulateur par Litmus Edge sans réécrire les tableaux de bord.

Architecture fonctionnelle de GlassInk Monitor

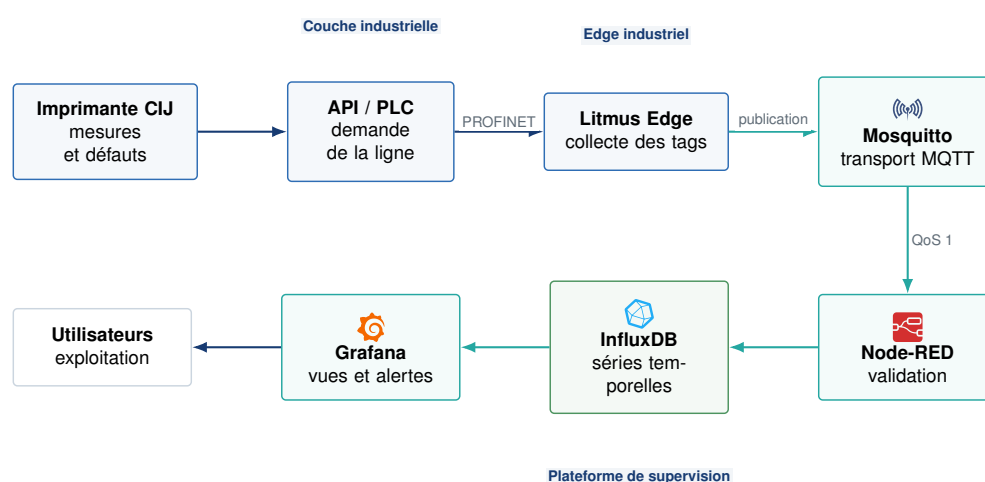


FIGURE 5.1 – Architecture fonctionnelle de GlassInk Monitor

Le PLC et l'imprimante appartiennent à la couche de contrôle existante. Litmus Edge fournit le point d'intégration. Mosquitto transporte les messages. Node-RED vérifie leur structure, les convertit en Line Protocol et les écrit dans InfluxDB. Grafana interroge ensuite le bucket de supervision.

5.2 Choix de la pile technique



5.2.1 MQTT et Mosquitto

MQTT suit le modèle publication/abonnement. Le producteur publie sur un topic, le courtier distribue le message et les consommateurs s'abonnent aux topics utiles. Mosquitto a été retenu parce que le volume du projet reste faible et qu'il fournit les fonctions nécessaires : authentification, ACL, TLS, persistance et outils de test [4].

Le prototype utilise le QoS 1. Ce niveau garantit une livraison "au moins une fois" [3]. Il améliore la fiabilité, mais autorise un doublon lors d'une retransmission. L'application doit donc être idempotente.

5.2.2 Node-RED

Node-RED rend le chemin des données visible dans un éditeur. Ce choix est adapté au passage du simulateur vers Litmus Edge : le mapping peut être inspecté et modifié sans cacher le traitement dans un service opaque. Les flux restent versionnés dans `flows.json`. L'éditeur est authentifié et les connexions utilisent les variables d'environnement [9].

5.2.3 InfluxDB

La plateforme produit surtout des données horodatées : mesures continues, changements d'état, défauts et résultats de marquage. InfluxDB 2.7 les conserve dans le `bucketprinter_monitoring`. Trois mesures structurent le contenu : `printer_telemetry`, `printer_event` et `marking_event`. Les identifiants utilisés pour filtrer deviennent des tags ; les valeurs numériques et descriptives restent des champs [12].

5.2.4 Grafana

Grafana interroge nativement InfluxDB. Il affiche les séries temporelles, les tableaux et les indicateurs calculés avec Flux. Les règles d'alerte réutilisent les mêmes données [15]. Les tableaux de bord et la source de données sont provisionnés par fichiers, ce qui évite une configuration manuelle lors d'une nouvelle installation.

5.2.5 Docker Compose

Chaque composant tourne dans son conteneur. Le fichier Compose décrit les services, leurs réseaux, leurs volumes, leurs variables et leurs contrôles de santé. Cette approche reproduit le même environnement sur un poste de développement et sur une VM de démonstration [16].

5.3 Cycle de vie des données

Cycle de vie d'une donnée de supervision

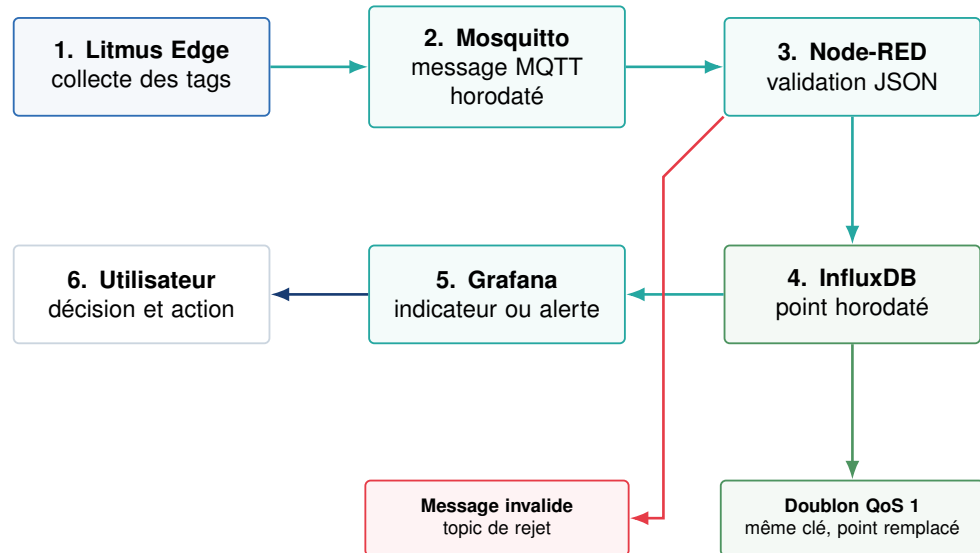


FIGURE 5.2 – Traitement d'une mesure entre la machine et le tableau de bord

Chaque message reçoit un `message_id`. Node-RED contrôle les champs obligatoires, construit un point InfluxDB puis l'envoie à l'API d'écriture v2. La clé d'un point associe la mesure, son jeu de tags et son horodatage. Une retransmission strictement identique réécrit donc le même point au lieu d'en créer un second [13]. Cette propriété couvre les doublons attendus avec le QoS 1. L'écriture de plusieurs mesures n'est toutefois pas une transaction globale ; en cas d'échec partiel, Node-RED rejoue les points avec les mêmes clés.

5.4 Topics MQTT

TABLE 5.1 – Arborescence MQTT du prototype

Type	Topic
Télémétrie	<code>sgx/<site>/<ligne>/printer/<id>/telemetry</code>
Événement	<code>sgx/<site>/<ligne>/printer/<id>/event</code>
Marquage	<code>sgx/<site>/<ligne>/printer/<id>/marking</code>
Rejet	<code>sgx/system/dead-letter</code>
État Node-RED	<code>sgx/system/node-red/status</code>

Lors du raccordement réel, Litmus Edge pourra publier du JSON dans cette arborescence ou utiliser Sparkplug B. Dans le second cas, un nœud de décodage protobuf sera ajouté avant la validation ; le stockage et les tableaux de bord ne changeront pas.

5.5 Modèle de données

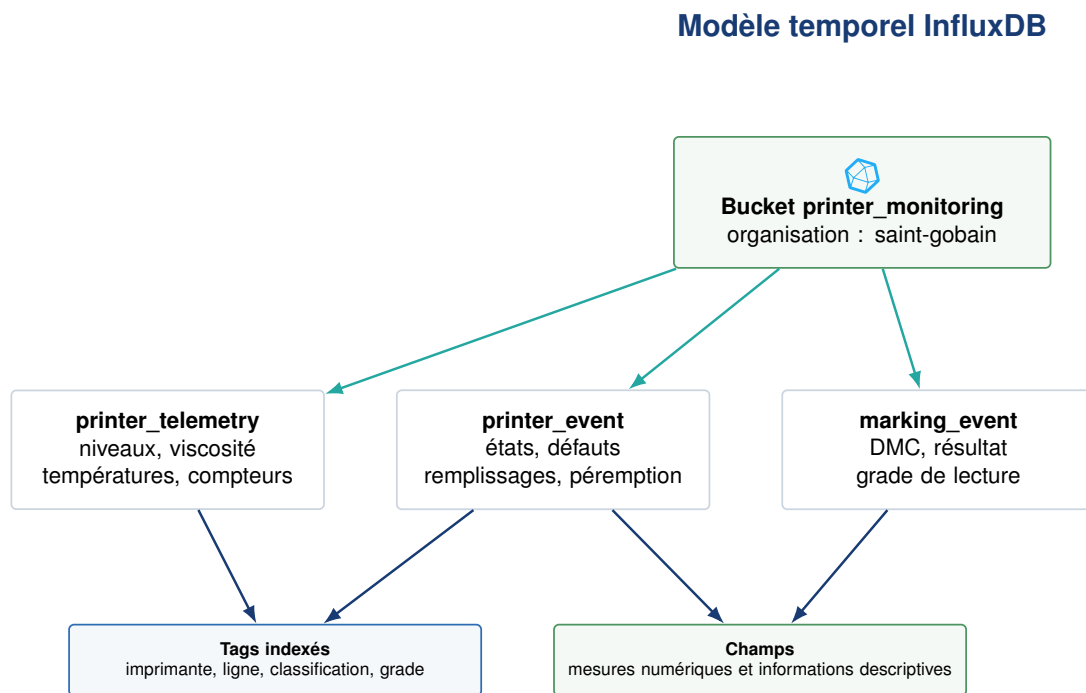


FIGURE 5.3 – Organisation des données dans le bucket InfluxDB

La mesure `printer_telemetry` regroupe les valeurs continues sous forme de champs : niveau d'encre, niveau de solvant, viscosité, températures, pression et compteurs. `printer_event` conserve les changements d'état, les défauts et les remplissages. `marking_event` contient le DMC, le résultat du contrôle et le grade de lecture. Les tags `printer_id`, `line_id`, `event_type` et `verify_grade` permettent les filtres courants sans transformer chaque valeur numérique en série indexée. Node-RED ajoute aussi le tag `classification` et les champs dérivés `hours_to_empty`, `action_days` et `expiry_limited` pour chaque fluide.

5.6 Disponibilité liée à la demande

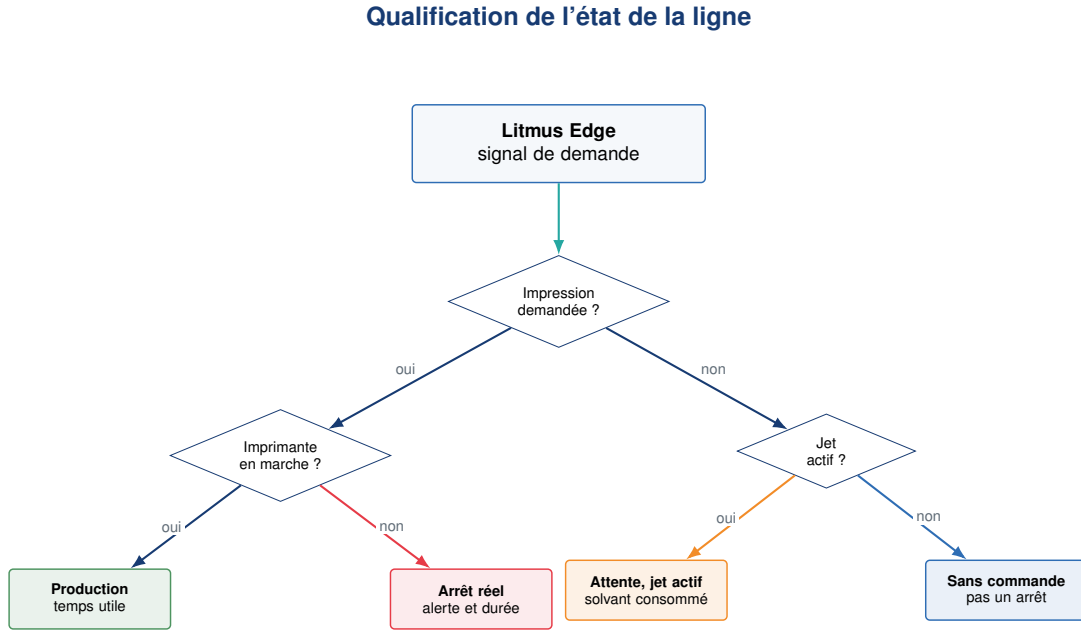


FIGURE 5.4 – Distinction entre production, arrêt et attente normale

La disponibilité est calculée sur les échantillons où la ligne demande une impression :

$$A = \frac{N_{\text{demande et marche}}}{N_{\text{demande}}} \times 100 \quad (5.1)$$

Une période sans commande n'entre ni au numérateur ni au dénominateur. Si le flux réel est publié à intervalle fixe, ce comptage approche correctement le temps. Si Litmus publie uniquement lors d'un changement, il faudra reconstruire des intervalles et pondérer par leur durée.

5.7 Prédiction et péremption

Node-RED estime la consommation à partir de deux niveaux successifs. Pour limiter l'effet du bruit, le débit observé est lissé avec le débit précédent :

$$r_k = 0,75 r_{k-1} + 0,25 \frac{L_{k-1} - L_k}{t_k - t_{k-1}} \quad (5.2)$$

Pour un débit r_k exprimé en points de pourcentage par heure et un niveau courant L_k , le temps avant épuisement vaut :

$$T_{\text{vide}} = \frac{L_k}{r_k} \quad (5.3)$$

La date d'action n'est pas forcément déterminée par ce calcul. Un fluide ouvert peut expirer avant d'être vide. La date effective est :

$$D_{\text{effective}} = \min (D_{\text{imprimée}}, D_{\text{ouverture}} + N_{\text{jours après ouverture}}) \quad (5.4)$$

Une hausse supérieure à cinq points indique un remplissage et réinitialise l'estimation. Le tableau de maintenance affiche les deux échéances et retient celle qui arrive en premier.

Réalisation de la plateforme

6.1 Organisation du projet

Le dépôt est organisé par service. Chaque dossier contient sa configuration, son code et un fichier README.

Listing 6.1 – Organisation des principaux composants

```
1 .
2 |-- broker/ # Eclipse Mosquitto
3 |-- dashboards/ # provisioning et tableaux Grafana
4 |-- simulator/ # modele Python de l'imprimante
5 |-- ingestion/ # image et flux Node-RED
6 |-- notifier/ # diffusion des alertes
7 |-- docker-compose.yml # services, volumes et initialisation InfluxDB
8 '-- Makefile
```

Les services de base démarrent sans profil. Le simulateur, Node-RED et le notifier utilisent des profils Compose. La commande `make demo` construit et lance l'ensemble.

6.2 Simulateur de l'imprimante CIJ

Le simulateur est écrit en Python. Il contient un état interne, deux consommables et un générateur de commandes. Son cycle de fonctionnement suit les états `shutdown`, `warmup`, `running`, `stopped` et `fault`. Une loi de Poisson simplifiée déclenche des défauts à une fréquence configurable.

Le modèle reprend les données documentées pour la 9450c : cartouches externes de 800 mL, réservoirs internes de 1,075 L, état du jet, état d'impression, vitesse moteur, pression, viscosité, températures, niveaux et autonomie d'encre. Les codes natifs sont publiés en parallèle de l'état fonctionnel utilisé par le tableau de bord. Par exemple, le code jet `00h` représente l'arrêt, `01h` le démarrage et `07h` le fonctionnement normal ; l'état d'impression distingue pause, impression opérationnelle et imprimante non prête.

TABLE 6.1 – Comportements reproduits par le simulateur

Comportement	Utilité pour les essais
Périodes de commande et périodes vides	Vérifier la disponibilité liée à la demande.
Consommation d'encre en production	Tester les tendances et la prévision d'épuisement.
Évaporation du solvant avec le jet actif	Éviter une prévision basée uniquement sur le nombre de pièces.
Remplissage avec saut de niveau	Vérifier le changement de pente et la gestion des lots.
Dérive de viscosité	Déclencher une alerte avant une dégradation visible.
Paramètres natifs de la 9450c	Vérifier le mapping futur des trames constructeur vers Litmus Edge.
Défauts avec code et durée	Alimenter l'historique, le temps d'arrêt et le Pareto.
Silence temporaire du producteur	Tester l'alerte d'absence de données.

Chaque message contient un UUID dans `message_id`. Le Last Will MQTT annonce aussi un arrêt si le client disparaît sans fermeture propre.

6.3 Ingestion avec Node-RED

Chaîne d'ingestion Node-RED

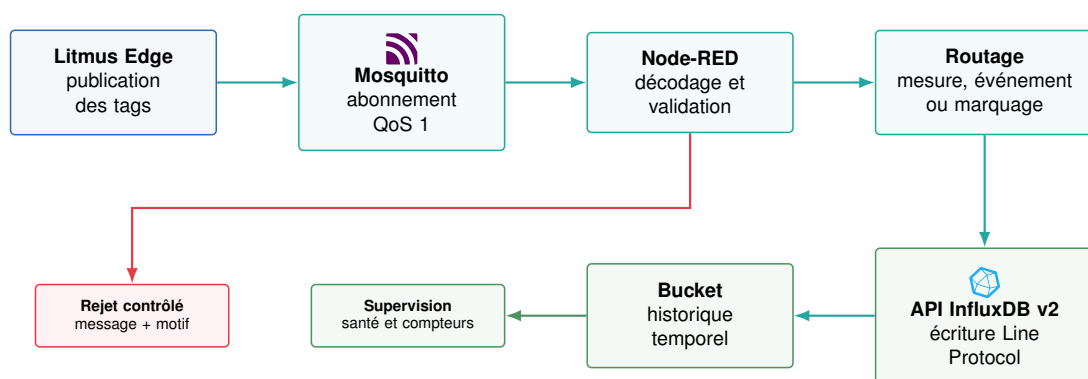


FIGURE 6.1 – Chaîne d'ingestion mise en œuvre avec Node-RED

Node-RED contient trois flux de données et un flux de supervision :

1. **Télémétrie** reçoit les valeurs continues, contrôle le payload et construit un point `printer_telemetry`.

2. **Événements** traite les changements d'état, défauts, remplissages et annonces de démarrage.
3. **Marquages** enregistre le DMC et le résultat de contrôle.
4. **Supervision** sonde InfluxDB, expose /health et centralise les erreurs.

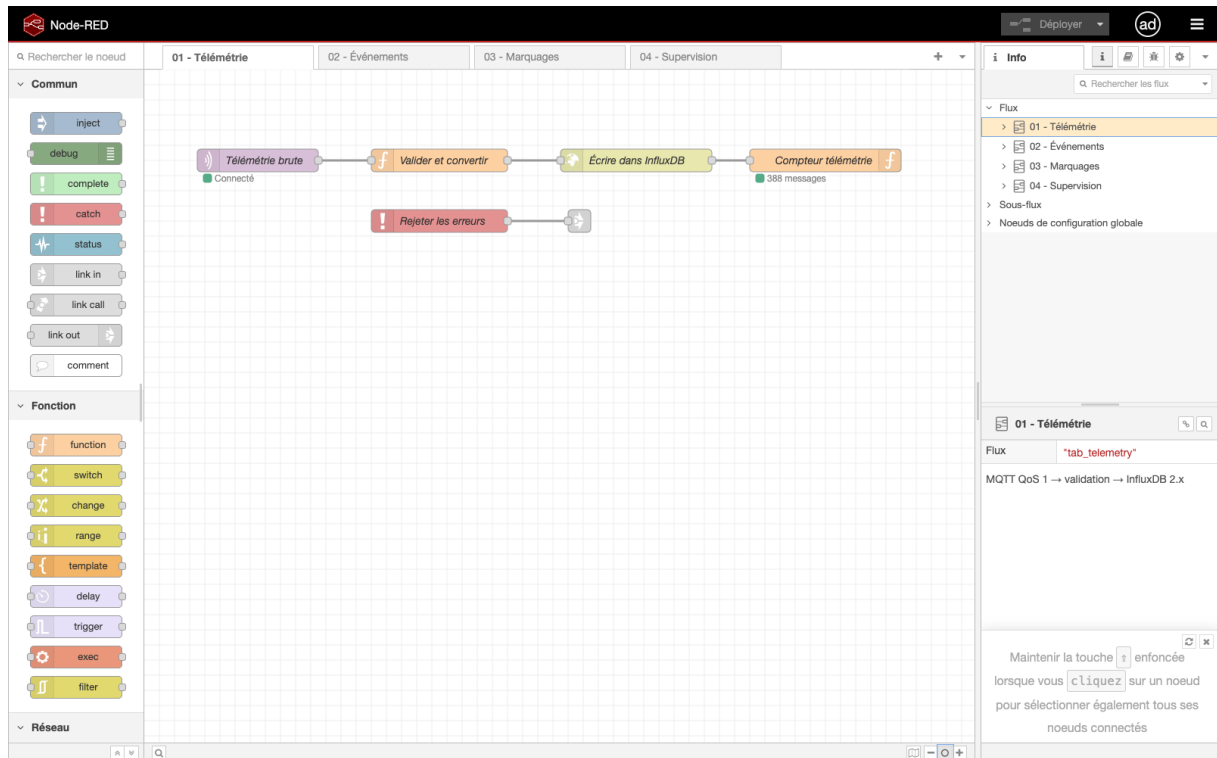


FIGURE 6.2 – Flux Node-RED de télémétrie : MQTT, validation et écriture InfluxDB

Node-RED utilise un nœud HTTP natif pour appeler directement l'API d'écriture v2 d'InfluxDB. Aucun module de base de données supplémentaire n'est nécessaire. Le token, le nom de l'organisation et le bucket proviennent des variables d'environnement; ils ne sont pas inscrits dans `flows.json`. Le nœud de préparation échappe les tags et les chaînes avant de produire le Line Protocol :

Listing 6.2 – Exemple de Line Protocol produit par Node-RED

```
1 printer_telemetry,printer_id=CIJ_Printer_L1,line_id=L1 \
2 ink_level=74.1,solvent_level=57.3,ink_viscosity=4.19 17854452000000
```

L'appel HTTP utilise une précision en millisecondes. Il cible l'organisation `saint-gobain` et le bucket `printer_monitoring`. Un message invalide est publié sur le topic de rejet avec son topic d'origine et un motif compréhensible par l'exploitant.

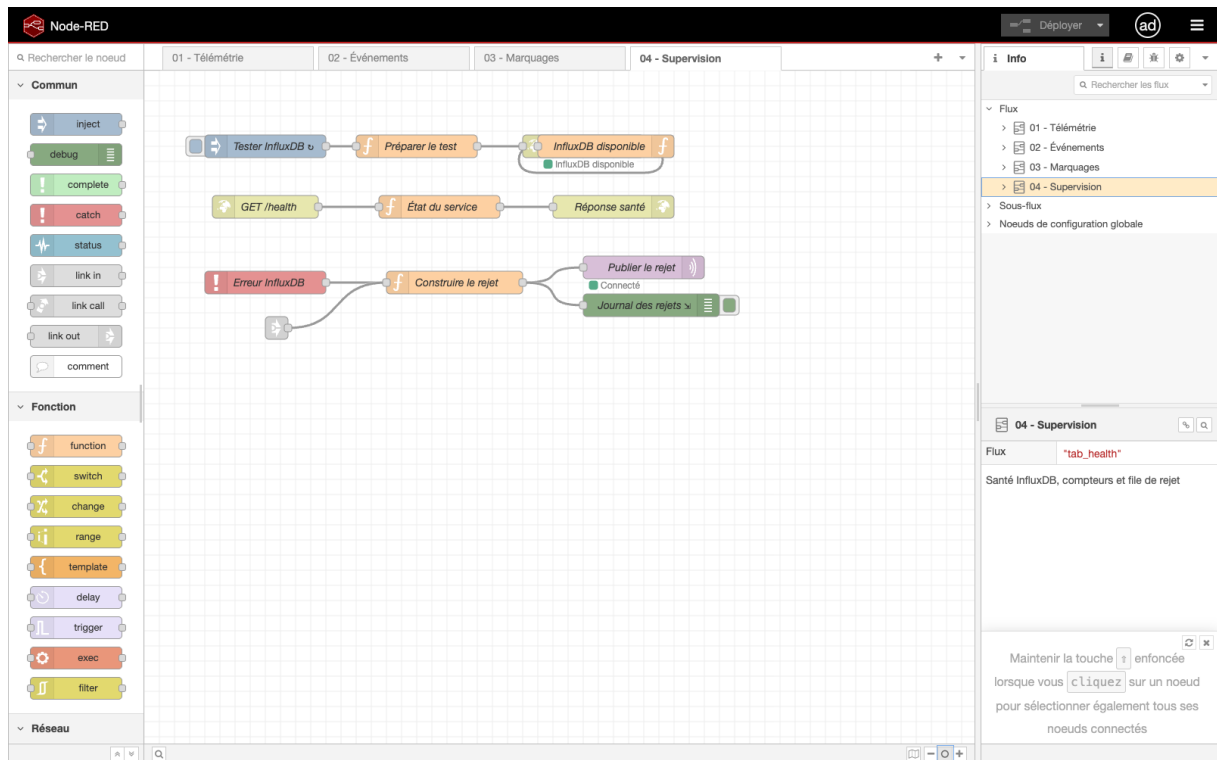


FIGURE 6.3 – Flux Node-RED de supervision et de traitement des rejets

6.4 Écriture idempotente et reprise

InfluxDB identifie un point par sa mesure, ses tags et son horodatage. Le simulateur conserve l'horodatage d'origine lors d'une retransmission. Node-RED reconstruit donc la même clé et InfluxDB met à jour le point existant au lieu de dupliquer l'observation [13]. Le message_id reste présent dans les événements pour faciliter le diagnostic, sans être utilisé comme tag sur la télémétrie afin de limiter la cardinalité.

Une écriture InfluxDB n'offre pas de transaction entre plusieurs mesures. Le flux traite chaque message de manière indépendante et conserve sa clé lors d'un nouvel essai. Cette limite est acceptable ici : les messages MQTT sont autonomes et le rejouage d'un point ne change pas le résultat final.

6.5 Requêtes Flux

Les calculs partagés sont écrits en Flux dans les tableaux de bord et les règles d'alerte. Ils couvrent notamment :

- la dernière valeur de chaque champ de télémétrie ;
- la lecture de la classification calculée par Node-RED à partir de la demande et de l'état machine ;
- la lecture des prévisions d'épuisement et de péremption ajoutées au point de télémétrie ;
- la disponibilité quotidienne, calculée uniquement pendant la demande ;

- le Pareto des défauts et le rendement au premier passage.

Les scripts Flux restent dans les fichiers JSON provisionnés de Grafana. Ils sont contrôlés par la commande `make verify`, qui exécute les requêtes contre le bucket de démonstration.

6.6 Conteneurisation

Docker Compose démarre six conteneurs : Mosquitto, InfluxDB, Grafana, le simulateur, Node-RED et le notifier. Les services attendent la santé de leurs dépendances. Les données du courtier, d’InfluxDB et de Grafana sont placées dans des volumes.

Les mots de passe sont générés dans un fichier `.env` exclu du dépôt. Le déploiement expose uniquement les interfaces nécessaires aux utilisateurs ; les services de transport et de stockage restent internes à la plateforme.

6.7 Notification des alertes

Grafana envoie toutes les alertes vers un webhook unique. Le notifier transforme le payload technique en une phrase courte, puis le transmet aux canaux activés. Les adaptateurs disponibles sont la console, Telegram, le courrier électronique, Microsoft Teams et une passerelle WhatsApp de démonstration.

L’adaptateur `open-wa` n’est pas retenu pour la production. Il pilote WhatsApp Web, dépend d’une session de téléphone et peut cesser de fonctionner après une mise à jour. Pour une utilisation industrielle, Teams, l’e-mail ou l’API WhatsApp Business officielle sont plus faciles à maintenir.

Tableaux de bord et alertes

7.1 Principes de présentation

Un seul tableau de bord pour tous les utilisateurs aurait accumulé les indicateurs jusqu'à devenir illisible. Trois vues ont donc été créées. Leur vocabulaire est en français et leurs requêtes renvoient un état explicite lorsqu'aucune donnée n'est encore disponible.

7.2 Vue opérateur

La vue opérateur doit être comprise rapidement, même à distance. Elle contient l'état de l'imprimante, le dernier défaut, les niveaux d'encre et de solvant, le délai avant action et le nombre de pare-brise marqués par période de cinq minutes.

TABLE 7.1 – Correspondance entre état technique et texte affiché

Classification	Libellé opérateur
producing	EN PRODUCTION
downtime	À L'ARRÊT
idle_jet_on	EN ATTENTE
idle_no_order	SANS COMMANDE
absence de mesure	AUCUNE DONNÉE

Le dernier défaut est traduit en français dans la requête. S'il n'existe aucun défaut, le panneau affiche "Aucun défaut" au lieu de "No data".



FIGURE 7.1 – Tableau de bord opérateur alimenté par la simulation

7.3 Vue maintenance

La vue maintenance rassemble les informations nécessaires à un diagnostic :

- codes natifs du jet et de l'impression, vitesse moteur, pression et viscosité générés par le simulateur 9450c;
- capacités de 800 mL et 1,075 L, réserve interne de 24 heures et températures électronique, encre et tête;
- tableau des consommables avec lot, niveau, jours avant épuisement, date effective de péremption et facteur limitant;
- courbes d'encre et de solvant;
- viscosité comparée à la consigne;
- pression et température de la tête;
- heures restantes avant service;
- liste des défauts récents;
- Pareto du temps perdu par code de défaut;
- historique des lots installés.

Le tableau des consommables met côte à côte les deux échéances. Une cartouche peut être encore remplie à 60 % et devoir être remplacée cinq jours plus tard à cause de

sa date après ouverture. Afficher uniquement le niveau masquerait ce cas.

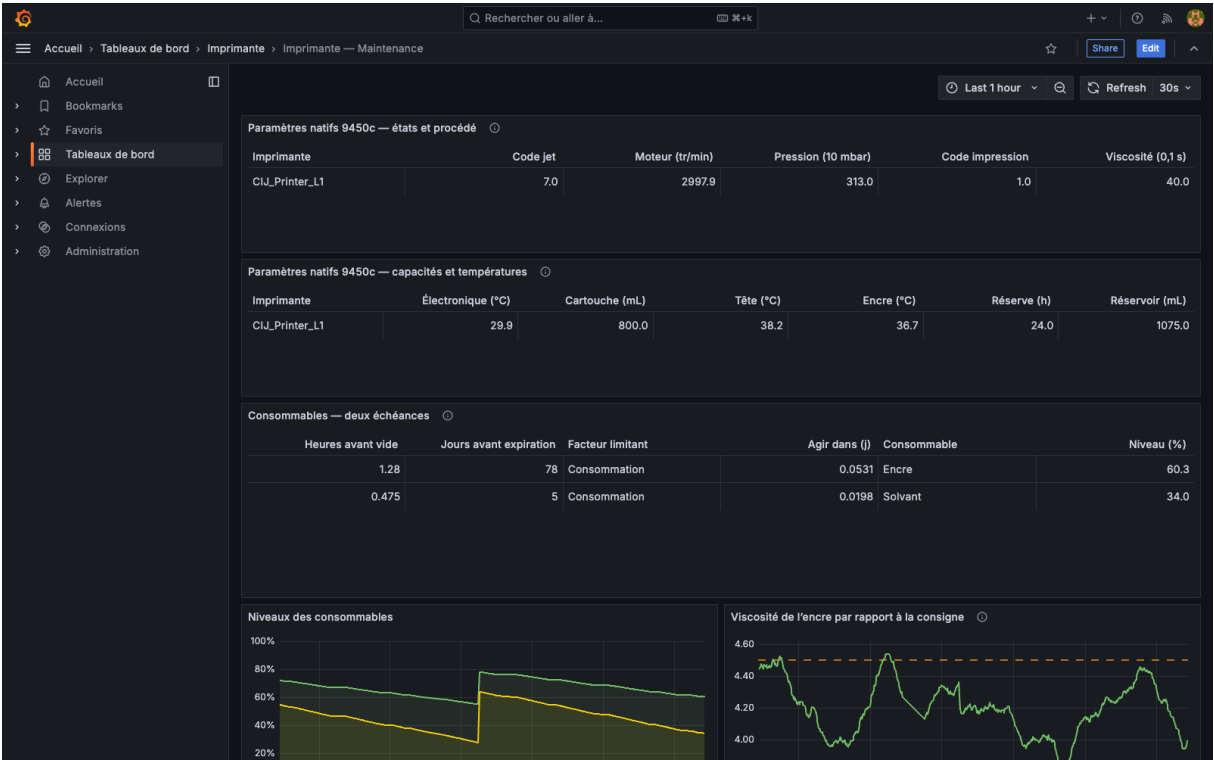


FIGURE 7.2 – Paramètres natifs produits en direct par la simulation 9450c

7.4 Vue direction

La vue direction évite les mesures brutes. Elle présente la disponibilité du jour, le nombre de pare-brise marqués, le rendement au premier passage, le temps d’arrêt, la répartition du temps et les principales causes d’arrêt. Le graphique de répartition conserve une catégorie “sans commande”, car cette information explique la différence entre temps calendrier et temps demandé.

Si aucune demande n’a encore été observée dans la journée, le panneau de disponibilité affiche “NON CALCULÉE”. Une valeur de 0 % aurait une autre signification : la ligne aurait demandé des impressions sans aucune production.

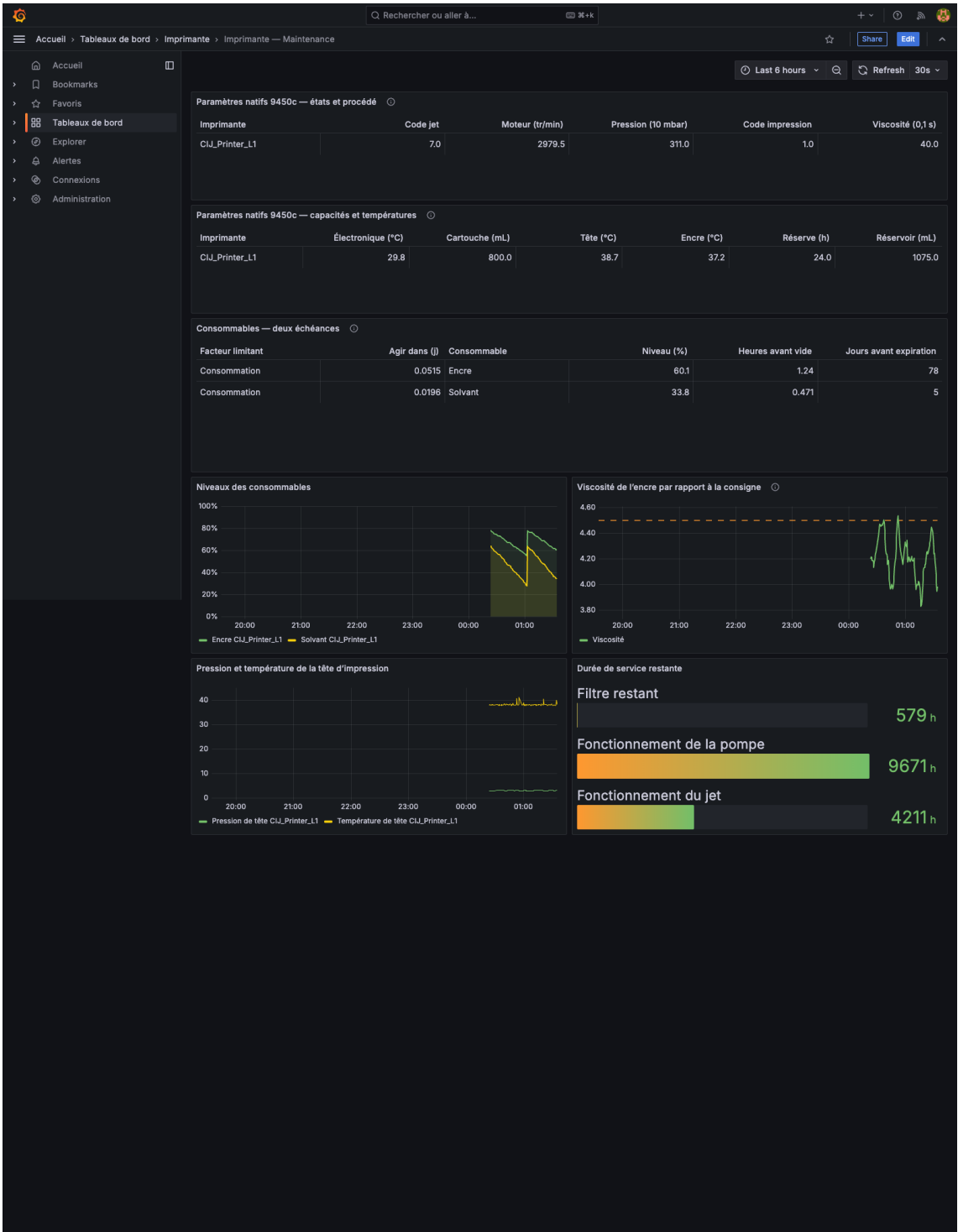


FIGURE 7.3 – Tableau de bord de maintenance alimenté par la simulation 9450c

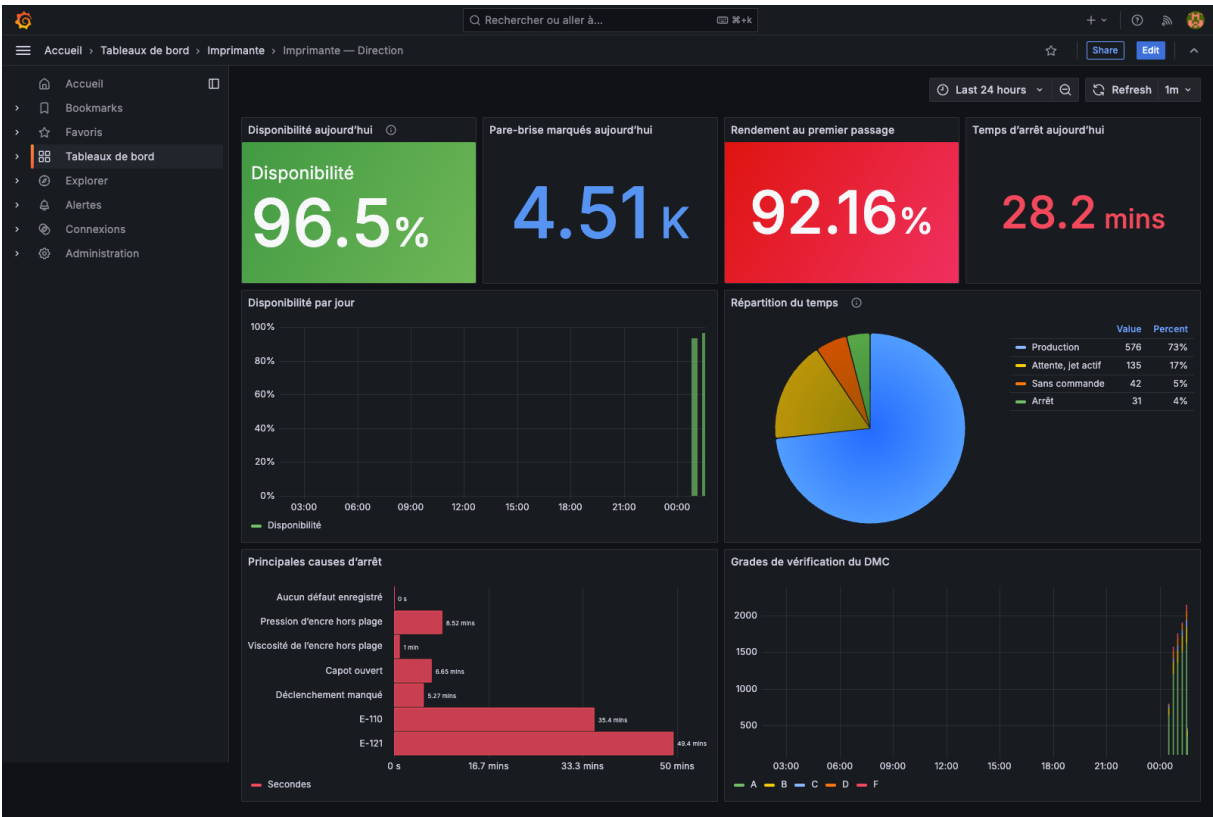


FIGURE 7.4 – Tableau de bord de direction alimenté par les données simulées

7.5 Règles d’alerte

TABLE 7.2 – Règles provisionnées dans Grafana

Alerte	Condition	Destinataire
Consommable bientôt vide	Temps estimé inférieur à 8 heures pendant 5 minutes	Maintenance
Niveau bas	Niveau inférieur à 20 % pendant 5 minutes	Maintenance
Consommable proche de la péremption	Moins de 14 jours avant la date effective	Maintenance
Consommable périmé	Date effective dépassée	Maintenance et qualité
Arrêt avec demande	Classification downtime pendant 2 minutes	Opérateur
Viscosité hors plage	Écart supérieur à 0,3 pendant 15 minutes	Maintenance
Absence de données	Dernière mesure reçue depuis plus de 120 secondes	Ingénierie

Les alertes liées à l'arrêt utilisent le signal de demande. En revanche, l'absence totale de télémétrie reste un incident, même sans commande : elle peut indiquer une perte de communication ou un service arrêté.

7.6 Lisibilité et langue

Les titres et les états visibles ont été traduits en français. Les noms de mesures, champs et codes de défaut restent techniques, car ils servent de contrat avec l'ingestion. La traduction intervient au niveau de la requête ou de l'affichage, sans modifier la donnée brute.

Les figures 7.1 à 7.4 présentent les vues de l'instance Grafana de démonstration alimentée par les données InfluxDB. Les panneaux disposent d'un état de repli explicite afin de ne pas confondre une absence de défaut avec une absence de données.

Validation du prototype

8.1 Stratégie de test

La validation combine des tests unitaires et des essais sur la plateforme complète. Les tests unitaires vérifient les règles du simulateur et le format des notifications. Les essais d'intégration publient de vrais messages dans Mosquitto, passent par Node-RED et contrôlent les points écrits dans InfluxDB. Enfin, les requêtes Flux de chaque panneau sont exécutées par l'API de Grafana.

8.2 Tests unitaires

Douze tests automatisés sont présents :

- huit tests du simulateur : identité des messages, date effective de péremption, consommation en attente, capacités des fluides, codes et paramètres 94xx, débit de marquage et absence de marquage sans demande ;
- quatre tests du notifier : format d'une alerte active, regroupement, résolution et protection contre un template Grafana non résolu.

La commande `make verify` exécute ces tests, vérifie la syntaxe Compose, le JSON des flux Node-RED et les fichiers des tableaux de bord.

8.3 Essais d'intégration

TABLE 8.1 – Résultats des essais de bout en bout

Essai	Observation	Résultat
Publication simulée	Télémétrie, événements et marquages présents dans le bucket.	Conforme
Message QoS 1 publié deux fois	Le nombre de points reste inchangé après la retransmission exacte.	Conforme
Payload sans message_id	Message reçu sur <code>sgx/system/dead-letter</code> avec le motif.	Conforme
Redémarrage Node-RED	Reconnexion MQTT puis reprise des écritures.	Conforme
Authentification Node-RED	<code>/flows</code> renvoie HTTP 401 sans session; les quatre onglets sont accessibles après connexion.	Conforme
Source Grafana	Connexion InfluxDB déclarée saine et bucket accessible.	Conforme
Tableaux de bord	23 requêtes de panneaux exécutées avec succès.	Conforme
Alertes	Sept règles chargées depuis les fichiers de provisioning.	Conforme

Au terme des essais, les six conteneurs étaient déclarés sains. Les trois mesures InfluxDB contenaient des points issus du simulateur, et la répétition d'un message conservait le même résultat. Le prototype valide donc la chaîne simulateur–MQTT–Node-RED–InfluxDB–Grafana.

8.4 Cas de la disponibilité

Le simulateur alterne les périodes de commande et les périodes vides. Les échantillons sont classés dans quatre catégories : production, arrêt réel, attente avec jet actif et absence de commande. Le tableau de bord ne compte que la deuxième catégorie comme arrêt.

Cet essai a révélé une limite connue. La requête quotidienne compte des échantillons et suppose une publication périodique. Ce calcul est correct pour le simulateur. Si Litmus Edge publie seulement lors d'un changement de valeur, les états longs et courts auraient le même poids. La validation sur site devra donc vérifier le mode de publication.

8.5 Cas des consommables

Le lot de solvant initial est ouvert depuis 85 jours, avec une durée configurée de 90 jours après ouverture. Sa date imprimée est plus éloignée, mais la requête calcule cinq jours restants. Ce scénario vérifie que la plateforme retient la première échéance.

Les remplissages produisent un saut de niveau. Lorsque la hausse dépasse cinq points, Node-RED réinitialise le débit estimé. Le délai avant épuisement reste alors vide jusqu'à la réception d'une nouvelle baisse mesurable, au lieu d'afficher une prévision négative ou héritée de l'ancienne cartouche.

Les tests de conformité 94xx contrôlent aussi la capacité de 800 mL des cartouches externes, la capacité de 1,075 L des réservoirs internes et les codes du jet. Un état running avec demande produit ainsi un code jet 7 et un état d'impression 1, tandis que la phase de démarrage produit le code jet 1.

8.6 Limites de la validation

Le prototype ne prouve pas encore la compatibilité avec les tags réels de l'imprimante. Il ne mesure pas non plus la latence du réseau industriel ni la qualité d'un payload Sparkplug B fourni par le site. Les défauts simulés sont plausibles, mais leurs codes ne remplacent pas le manuel du modèle installé.

Ces limites ne remettent pas en cause les composants en aval. Elles définissent le travail de raccordement : capturer un payload réel, établir la table de mapping, vérifier les unités et rejouer les tests d'acceptation.

Sécurité et préparation du déploiement

9.1 Constat réseau

Les informations disponibles indiquent qu’aucun pare-feu ne sépare actuellement la plateforme envisagée du réseau des automates. Cette situation augmente le risque : une erreur de configuration pourrait rendre un service accessible depuis une zone qui n’en a pas besoin.

Déploiement et limites réseau

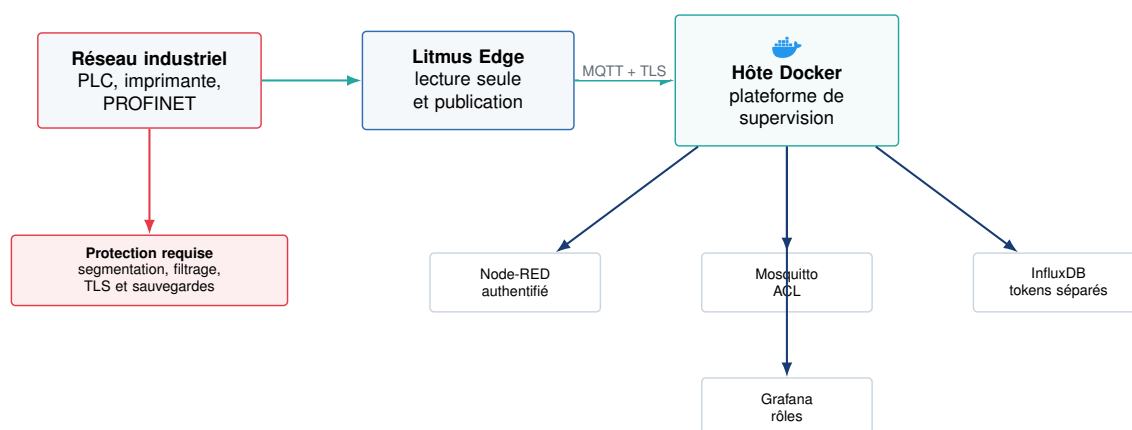


FIGURE 9.1 – Déploiement proposé et mesures à appliquer avant le raccordement

9.2 Mesures intégrées au prototype

- les services du projet ne commandent aucun équipement industriel ;
- Node-RED, InfluxDB, Mosquitto et le notifieur sont liés à l’interface locale dans l’environnement de développement ;
- l’éditeur Node-RED demande un identifiant et un mot de passe ;
- les secrets restent dans `.env`, avec des permissions de fichier limitées ;
- Mosquitto possède un exemple de configuration TLS avec comptes séparés et ACL ;

- le flux Node-RED ne contient pas le token InfluxDB;
- les messages invalides sont isolés dans un journal de rejet.

9.3 Mesures requises sur site

Avant toute connexion à Litmus Edge, plusieurs actions sont nécessaires :

1. placer la VM de supervision dans un VLAN défini avec le responsable du réseau industriel;
2. appliquer un pare-feu hôte avec une politique de refus par défaut;
3. autoriser uniquement les connexions nécessaires entre Litmus Edge, le courtier et les interfaces utilisateur;
4. activer TLS pour MQTT et utiliser un certificat délivré ou approuvé par l'entreprise;
5. créer un compte MQTT par client et limiter ses topics;
6. créer un token InfluxDB en lecture seule pour Grafana et un token d'écriture distinct pour Node-RED;
7. intégrer Grafana à l'identité d'entreprise si elle est disponible;
8. sauvegarder le volume InfluxDB et tester une restauration sur une machine distincte.

9.4 Politique MQTT

Litmus Edge doit pouvoir publier dans son espace de topics. Node-RED doit lire ces topics et publier uniquement les rejets et son état. Grafana n'a pas besoin d'accéder au courtier. Cette séparation réduit les droits de chaque service.

Listing 9.1 – Extrait d'ACL MQTT de production

```
1 user litmus_edge
2 topic write sgx/<site>/#
3
4 user node_red
5 topic read sgx/<site>/#
6 topic write sgx/system/dead-letter
7 topic write sgx/system/node-red/status
```

9.5 Données et conservation

Le volume annoncé, inférieur à 200 pièces par heure sur une ligne, reste modeste. Le prototype conserve tous les événements et compresse les mesures anciennes. L'entreprise n'a pas encore fourni de durée officielle de conservation. Cette durée devra être définie avec la qualité et la sécurité des systèmes d'information avant la production.

Les sauvegardes doivent couvrir le volume InfluxDB, les volumes Grafana utiles et les fichiers de configuration versionnés. Une sauvegarde qui n'a jamais été restaurée ne constitue pas une preuve de reprise.

Conclusion générale

GlassInk Monitor transforme des données industrielles dispersées en une chaîne de supervision cohérente. Le prototype reçoit les messages MQTT, les valide dans Node-RED, les convertit en Line Protocol, les historise dans InfluxDB et les présente dans trois tableaux de bord Grafana. Il suit l'état de l'imprimante, les consommables, les défauts et le résultat des marquages.

Le point le plus délicat n'a pas été l'affichage d'une jauge. Il a été de définir ce qu'est réellement un arrêt. Sur une ligne qui fonctionne selon les commandes, une machine immobile n'est pas forcément indisponible. L'ajout d'un signal de demande rend les indicateurs et les alertes crédibles. La comparaison entre épuisement et péremption répond au même principe : un pourcentage seul ne suffit pas pour décider quand intervenir.

Le recours au simulateur a permis d'avancer sans accès au Litmus Edge. Il a fourni des périodes sans commande, des défauts, des remplissages et des pertes de communication. Les essais ont validé les doublons MQTT, le journal de rejet, la reprise après redémarrage, les 23 requêtes de tableaux de bord et les sept règles d'alerte.

Le projet n'est pas encore raccordé à la production. La prochaine étape est précise : obtenir un payload Litmus et la liste des tags, confirmer le modèle de l'imprimante et les unités, puis remplacer le mapping du simulateur. Cette phase devra aussi valider la sécurité du réseau, le TLS, les ACL, les comptes Grafana et la sauvegarde.

Ce PFA m'a permis de relier plusieurs domaines de ma formation : réseaux industriels, messagerie IoT, bases de données temporelles, conteneurisation et supervision. Il m'a surtout appris qu'un indicateur industriel n'est utile que si sa définition correspond au fonctionnement réel de la ligne.

Contrats de messages

A.1 Exemple de télémétrie

Listing A.1 – Payload de télémétrie publié par le simulateur

```
1 {  
2   "message_id": "4efc62a318f84e36bb78cc1341a88b3d",  
3   "ts": 17854452000000,  
4   "printer_id": "CIJ_Printer_L1",  
5   "line_id": "L1",  
6   "metrics": {  
7     "printer_state_code": 2,  
8     "jet_status_code": 7,  
9     "printing_status_code": 1,  
10    "jet_running": 1,  
11    "line_demanding": 1,  
12    "ink_level": 74.1,  
13    "solvent_level": 57.3,  
14    "ink_viscosity": 4.19,  
15    "head_pressure": 3.02,  
16    "head_temperature": 38.4,  
17    "motor_speed_rpm": 2998.0,  
18    "pressure_10mbar": 302,  
19    "viscosity_tenth_second": 42,  
20    "print_count": 184215  
21  }  
22 }
```

A.2 Exemple d'événement de remplissage

Listing A.2 – Payload créant un nouveau lot de solvant

```
1 {  
2   "message_id": "a72ba6534dc84b1a95ff5d182d653458",  
3   "ts": 17854452000000,  
4   "printer_id": "CIJ_Printer_L1",  
5   "event_type": "refill",
```

```
6  "consumable_type": "solvent",  
7  "part_number": "MI-9876-S",  
8  "batch_code": "LOT-B12",  
9  "manufactured_on": "2026-01-15",  
10 "expires_on": "2027-01-15",  
11 "opened_on": "2026-07-30",  
12 "shelf_life_after_open_days": 90,  
13 "volume_ml": 800  
14 }
```

Commandes d'exploitation

TABLE B.1 – Commandes principales du prototype

Commande	Effet
<code>make demo</code>	Construit et démarre les six services.
<code>make ps</code>	Affiche l'état et la santé des conteneurs.
<code>make logs</code>	Suit les journaux des services.
<code>make verify</code>	Exécute les tests et les contrôles d'acceptation.
<code>make influx</code>	Ouvre les informations et outils de contrôle InfluxDB.
<code>make mqtt-sub</code>	Affiche les messages de tous les topics.
<code>make down</code>	Arrête la plateforme sans supprimer les données.

Bibliographie

- [1] Saint-Gobain Sekurit, *Automotive Glass Manufacturing*, documentation sur les procédés de fabrication du vitrage automobile, <https://www.saint-gobain-sekurit.com/sites/hps-mac3-sekurit-sekurit/files/2021-04/Saint-Gobain%20Sekurit%20automotive%20glazing%20production%20processes.pdf>.
- [2] Markem-Imaje, *9450 Continuous Inkjet Printer – Technical Specifications*, fiche technique constructeur, <https://www.markem-imaje.com/docs/default-source/product-brochures/hq/9450-ds-hq-d1-markem-imaje.pdf>.
- [3] OASIS, *MQTT Version 5.0*, standard OASIS, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>.
- [4] Eclipse Mosquitto, *Documentation du courtier MQTT*, <https://mosquitto.org/documentation/>.
- [5] Litmus Automation, *Generic MQTT Integration Guide*, <https://docs.litmus.io/litmusedge/how-to-guides/integration-guides/generic-mqtt-tcp-lwt>.
- [6] Litmus Automation, *Litmus Edge API Documentation*, <https://api.litmus.io/le/>.
- [7] OPC Foundation, *OPC Unified Architecture – Part 1 : Overview and Concepts*, <https://reference.opcfoundation.org/specs/OPC-10000-1/4>.
- [8] Eclipse Foundation, *Sparkplug Specification 3.0.0*, <https://sparkplug.eclipse.org/specification/>.
- [9] Node-RED, *User Guide*, <https://nodered.org/docs/>.
- [10] InfluxData, *Telegraf Plugin Directory*, <https://docs.influxdata.com/telegraf/v1/plugins/>.
- [11] InfluxData, *InfluxDB OSS v2 Documentation*, <https://docs.influxdata.com/influxdb/v2/>.
- [12] InfluxData, *InfluxDB schema design*, <https://docs.influxdata.com/influxdb/v2/write-data/best-practices/schema-design/>.
- [13] InfluxData, *Handle duplicate points*, <https://docs.influxdata.com/influxdb/v2/write-data/best-practices/duplicate-points/>.
- [14] InfluxData, *Get started with Flux*, <https://docs.influxdata.com/influxdb/v2/query-data/get-started/>.
- [15] Grafana Labs, *Configure the InfluxDB data source*, <https://grafana.com/docs/grafana/latest/datasources/influxdb/configure-influxdb-data-source/>.
- [16] Docker, *Multi-container applications with Docker Compose*, <https://docs.docker.com/get-started/docker-concepts/running-containers/multi-container-applications/>.
- [17] Z. Li, F. Fei et G. Zhang, “Edge-to-Cloud IIoT for Condition Monitoring in Manufacturing Systems with Ubiquitous Smart Sensors”, *Sensors*, vol. 22, no 15, article 5901, 2022, <https://doi.org/10.3390/s22155901>.

- [18] C. Krupitzer et al., “A Survey on Predictive Maintenance for Industry 4.0”, 2020, <https://arxiv.org/abs/2002.08224>.